

Trabalho Prático 3

Consultas ao Sistema Logístico

Matheus Soares dos Santos de Freitas

Matrícula: 2024080043

matheussdsf@ufmg.br

1 Introdução

Este relatório apresenta a solução desenvolvida para o problema de consultas ao sistema logístico dos Armazéns Hanói, dando continuidade ao trabalho anterior. O sistema permite recuperar informações a partir de dois tipos principais de consultas: histórico de eventos de um pacote e histórico de pacotes associados a um cliente.

A implementação baseia-se em uma estrutura orientada a objetos, com uso de árvores AVL para buscas eficientes e um vetor dinâmico para armazenamento dos eventos em ordem cronológica.

O restante deste documento está organizado da seguinte forma: a Seção 2 descreve os métodos e estruturas utilizados; a Seção 3 analisa a complexidade dos algoritmos; a Seção 4 discute estratégias de robustez; a Seção 5 apresenta os resultados experimentais; e as Seções 6 e 7 trazem as conclusões e a bibliografia.

2 Método

A implementação do projeto foi estruturada de forma modular, com o objetivo de promover a clareza, a organização e a facilidade de manutenção do código-fonte. Os arquivos de cabeçalho foram alocados no diretório `include`, enquanto as definições das funções e classes correspon-

dentes encontram-se na pasta src. A raiz do projeto contém um arquivo Makefile, responsável pela automatização do processo de compilação. Os arquivos objeto gerados são armazenados na pasta obj, ao passo que o executável final é direcionado para o diretório bin, com o nome tp3.out.

2.1 Classes e Estruturas de Dados

A solução foi estruturada em torno de classes e estruturas que modelam pacotes, eventos, clientes e suas respectivas relações. As principais estruturas são descritas a seguir:

2.1.1 Classes Principais

- **Classe `arvoreAVL`**: árvore binária de busca balanceada, generalizada por template. Utilizada para armazenar chaves de pacote, tempo e clientes de forma eficiente. Dentre os métodos principais, destacam-se: *insere*, *busca*, *limpa*, *altura*, *fatorBalanceamento*, *atualizarAltura*, *rotacaoDireita* e *rotacaoEsquerda*. Também foram implementados métodos especializados para impressão e contagem de eventos: *imprimeInOrderPacoteAte*, *emOrdemApartirDe*, *contaEventosPacoteAte*, *contaEventosDeChegadaClienteAte*, *imprimeEventosDeChegadaClienteAte* e *imprimeEventosFinaisClienteAteInOrder*.
- **Classe `Evento`**: representa as diversas ocorrências do sistema logístico, como registros, transferências e entregas. Sua construção é sensível ao tipo do evento, o que determina os atributos inicializados. Os principais métodos são: *imprimeEvento*, *getTipoEvento*, *getDataHora* e *getIdPac*.
- **Classe `vetorEventos`**: armazena dinamicamente os eventos em ordem cronológica. Os métodos *insereEvento* e *acessaEvento* permitem inserir e acessar eventos, enquanto *aumentaCapacidade* cuida da realocação do vetor e *getQuantidade* retorna o número total de eventos armazenados.
- **Classe `Pacote`**: representa um pacote individual no sistema. Cada objeto possui um identificador e dois ponteiros para eventos: *inicio*, que aponta para o primeiro evento do pacote, e *fim*, que aponta para o evento mais recente. Os métodos principais incluem: *getId*, *getInicio*, *getFim* e *atualizaFim*.
- **Classe `noCliente`**: representa um cliente e os pacotes a ele associados, funcionando

como nó de uma árvore AVL. Cada cliente possui um nome, um ponteiro para uma lista ligada de pacotes (*CelulaIndice*) e um contador de pacotes. Os métodos incluem: *adicionaIndice*, *getListaIndices*, *getNome* e *getQntdPacotes*. Também são definidos os operadores $<$ e $>$, bem como construtor de cópia, destrutor e operador de atribuição.

2.1.2 Estruturas de Dados Básicas

- **Struct ChaveEvento:** abstrai a chave usada na árvore AVL para indexar eventos cronologicamente. Armazena os campos *idPacote*, *tempoEvento*, *ordemInsercao* e um ponteiro para o *Evento* correspondente. Implementa os operadores $<$, $==$, $>=$, $>$ e o operador de saída $\ll$, permitindo ordenação e impressão dos eventos.
- **Struct NoPacote:** nó auxiliar usado na árvore AVL para consultas por pacote. Contém os campos *idPacote* e *tempoRegistro* (momento do primeiro evento do pacote). Os operadores $<$, $>$, $==$ são implementados com base no campo *idPacote*.
- **Struct CelulaIndice:** representa um nó da lista encadeada de pacotes associada a um cliente. Cada célula armazena o *indicePacote*, um ponteiro para o *Pacote* correspondente e um ponteiro para a próxima célula. Essa estrutura é essencial para vincular múltiplos pacotes a um mesmo cliente, viabilizando as consultas por cliente.

2.2 Fluxo de Execução

A execução tem início no arquivo *main.cpp*, responsável por ler e armazenar as linhas do arquivo de entrada. Em seguida, cada linha é processada conforme seu tipo: evento logístico ou consulta.

Eventos (EV) são representados por objetos da classe *Evento* e indexados na AVL *avlChavesEventos* por meio de uma *ChaveEvento*, que associa tempo, pacote e ordem de inserção. Quando o evento é de registro (RG), é criada uma instância da classe *Pacote*, vinculada ao remetente e destinatário na AVL *avlClientes*, cujos nós (*noCliente*) armazenam listas encadeadas de pacotes.

Consultas do tipo CL (cliente) percorrem a AVL de clientes e, para cada pacote associado, imprimem eventos de chegada e final, respeitando o limite temporal da consulta. Já as consultas PC (pacote) buscam diretamente pelos eventos do pacote na AVL, também limitados por tempo.

Ao fim do processamento, as saídas são formatadas conforme as especificações do problema.

3 Análise de Complexidade

Considere $T(n)$ e $S(n)$, respectivamente, as complexidades de tempo e de espaço para o programa.

3.1 Classe **arvoreAVL**

A estrutura **arvoreAVL** garante altura balanceada $\mathcal{O}(\log n)$, o que assegura operações eficientes. Suas principais operações possuem os seguintes custos:

- `arvoreAVL()` : inicializa a árvore vazia.
 $T(n) = \mathcal{O}(1), S(n) = \mathcal{O}(1)$
- `Destructor`: percorre recursivamente todos os n nós e desaloca.
 $T(n) = \mathcal{O}(n), S(n) = \mathcal{O}(\log n)$
- `insere()` : percorre até a posição correta em $\mathcal{O}(\log n)$ e aplica no máximo duas rotações (constantes).
 $T(n) = \mathcal{O}(\log n)$ por inserção, $S(n) = \mathcal{O}(n)$ acumulado
- `busca()` : segue a estrutura da árvore balanceada.
 $T(n) = \mathcal{O}(\log n), S(n) = \mathcal{O}(\log n)$
- Métodos como `imprimeInOrderPacoteAte()` e `contaEventosPacoteAte()` : percorrem parcial ou totalmente a árvore.
 $T(n) = \mathcal{O}(n), S(n) = \mathcal{O}(\log n)$
- Métodos que utilizam listas ligadas (`contaEventosDeChegadaClienteAte()`): percorrem os nós e, em cada um, uma lista de até m elementos.
 $T(n) = \mathcal{O}(n \cdot m), S(n) = \mathcal{O}(\log n)$

Resumo: $T(n) = \mathcal{O}(n), S(n) = \mathcal{O}(n)$

3.2 Classe `vetorEventos`

Trata-se de um vetor dinâmico de ponteiros para **Evento**, com redimensionamento progressivo.

- `vetorEventos()`: aloca vetor de capacidade inicial c .

$$T(n) = \mathcal{O}(1), S(n) = \mathcal{O}(c)$$

- Destrutor: desaloca os n ponteiros e o array.

$$T(n) = \mathcal{O}(n), S(n) = \mathcal{O}(1)$$

- `insereEvento()`: em inserções comuns, o tempo é $\mathcal{O}(1)$; no redimensionamento (duplicação), ocorre cópia de todos os elementos. Como o redimensionamento acontece quando $n = 2^k$, temos:

$$T(n) = \sum_{i=0}^{\log n} 2^i = \mathcal{O}(n)$$

Assim, o tempo amortizado por inserção continua sendo $\mathcal{O}(1)$, com total $T(n) = \mathcal{O}(n)$.

$S(n) = \mathcal{O}(n)$ (capacidade sempre $\geq n$)

- Métodos `acessaEvento()` e `getQuantidade()`:

$$T(n) = \mathcal{O}(1), S(n) = \mathcal{O}(1)$$

Resumo: $T(n) = \mathcal{O}(n), S(n) = \mathcal{O}(n)$

3.3 Programa Principal

O programa processa um arquivo de entrada com k eventos e q consultas, distribuídas entre clientes (c) e pacotes (p). As estruturas manipuladas são três árvores AVL e um vetor dinâmico.

- `Eventos EV`: envolvem inserções nas árvores de chaves para eventos e de clientes. Como cada evento pode afetar ambas, temos:

$$T_{\text{eventos}}(k) = \mathcal{O}(k \cdot (\log k + \log c))$$

- `Consultas CL (clientes)`: percorrem a árvore AVL e, para cada nó, percorrem a lista de pacotes do cliente (tamanho m):

$$T_{\text{clientes}}(c) = \mathcal{O}(c \cdot k \cdot m)$$

- Consultas PC (pacotes): percorrem a árvore de eventos por pacote.

$$T_{\text{pacotes}}(p) = \mathcal{O}(p \cdot k)$$

- Espaço total:

$$S(n) = \mathcal{O}(k + c \cdot m)$$

devido à indexação de eventos e ao armazenamento de pacotes por cliente.

Complexidade total do programa:

$$T(n) = \mathcal{O}(k \log k + ck m + pk) \quad S(n) = \mathcal{O}(k + c \cdot m)$$

4 Estratégias de Robustez

O programa emprega diversas medidas de robustez para garantir sua estabilidade mesmo diante de entradas adversas ou inconsistentes. No nível do programa principal, há verificação da existência e abertura bem-sucedida do arquivo de entrada. As inserções nas estruturas de dados são acompanhadas de checagens contra duplicação e ponteiros nulos, especialmente ao manipular clientes e pacotes. A classe **vetorEventos** previne acessos inválidos ao vetor por meio da validação de índices e implementa realocação segura durante o redimensionamento. A classe **arvoreAVL**, por sua vez, garante o balanceamento automático após cada inserção e implementa destrutores recursivos seguros para liberação completa da memória. Tais práticas contribuem para a integridade da memória, previnem estouros de pilha e evitam comportamentos indefinidos em tempo de execução.

5 Análise Experimental

Para garantir maior precisão na análise experimental, o tempo total de execução foi dividido em duas etapas: leitura e processamento. Essa separação evita que fatores externos, como o desempenho de I/O da máquina, distorçam os resultados. As linhas do arquivo de entrada foram inicialmente armazenadas em memória usando um vetor auxiliar, com a medição dessa etapa registrada como tempo de leitura. Em seguida, o tempo de processamento foi cronometrado a partir dos dados já carregados, utilizando a biblioteca `<chrono>`.

Durante os testes, três dimensões foram analisadas individualmente, com medições sistemáticas dos tempos correspondentes. Os resultados são apresentados a seguir.

5.1 Impacto do Número de Pacotes no Tempo de Processamento

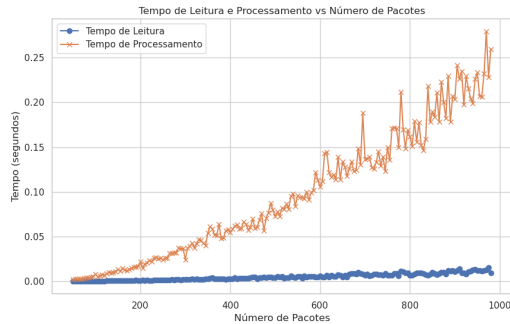


Figura 1: Tempo de processamento e leitura em função do número de pacotes.

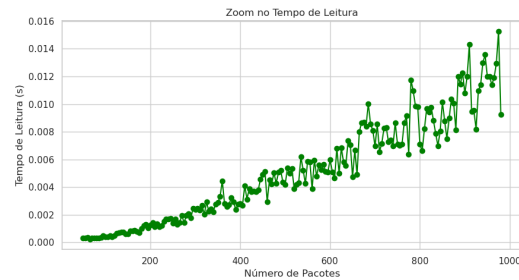


Figura 2: Tempo de leitura isolado em função do número de pacotes.

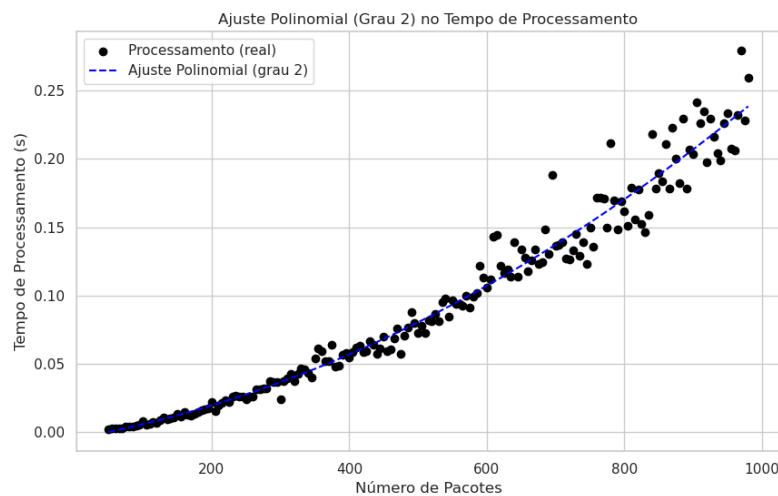


Figura 3: Tempo de processamento em função do número de pacotes.

A influência do número de pacotes no desempenho do programa é analisada nas Figuras 1, 2 e 3. Mesmo com as demais variáveis fixadas, observa-se que o tempo de *processamento* cresce de forma mais acentuada - quadrática - em relação ao de *leitura*. Para destacar essa diferença, a Figura 2 apresenta a curva de leitura em escala própria.

Conforme discutido na Seção 3, o número total de eventos k cresce proporcionalmente ao número de pacotes p , ou seja, $k = \alpha p$. Sabendo que a complexidade de consultas por pacote é da ordem $O(p \cdot k)$, tem-se:

$$T_{\text{pacotes}} = O(p \cdot \alpha p) = O(p^2)$$

Esse crescimento quadrático, ainda que teórico, justifica a tendência ascendente observada no terceiro gráfico de desempenho.

5.2 Impacto do Número de Clientes no Tempo de Processamento

Como apresentado na Figura 4, o tempo total de processamento diminui à medida que cresce o número de clientes no sistema. Para interpretar esse comportamento, retomamos a complexidade teórica das consultas do tipo CL, discutida na Seção 3.

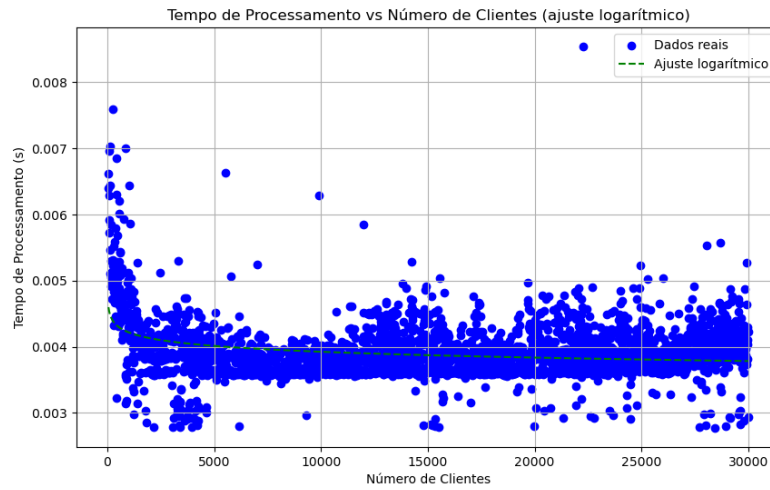


Figura 4: Tempo de processamento em função do número de clientes.

Considerando que o número de pacotes p e de consultas está fixo, o número médio de pacotes por cliente ($m = \frac{p}{c}$) tende a cair conforme c aumenta. Como o número total de eventos k é proporcional a p ($k = \alpha \cdot p$), temos:

$$T_{\text{CL}} = O(\text{numcl} \cdot k \cdot m) = O\left(\text{numcl} \cdot \frac{p^2}{c}\right).$$

Com p e o número de consultas fixos, o termo dominante no tempo total de processamento é inversamente proporcional ao número de clientes:

$$T_{\text{total}} = O\left(\frac{1}{c}\right).$$

Esse comportamento justifica a queda acentuada observada inicialmente na curva experimental, seguida de uma estabilização conforme c cresce. Conclui-se, assim, que a distribuição dos pacotes entre um número maior de clientes reduz a carga média por consulta, o que desacelera o tempo de processamento até um ponto de saturação.

5.3 Impacto do Número de Eventos no Tempo de Processamento

Para avaliar o impacto do número de eventos no desempenho do programa, considerou-se que tanto a contenção por pacotes quanto a topologia do sistema de armazéns são determinantes para a geração dos diferentes tipos de eventos presentes no sistema. Dessa forma, variações controladas dessas duas variáveis foram utilizadas nesta etapa da análise.

Tabela 1: Média de eventos processados por número de armazéns

Armazéns	Média de Eventos
2	597.38
3	630.84
4	721.14
5	829.90
6	786.32
7	1007.90
8	967.12
9	977.34
10	1067.62

A Tabela 1 ilustra a média de eventos gerados a cada iteração em função do número de armazéns no sistema. Observa-se que, à medida que o número de armazéns aumenta, há uma elevação no número médio de eventos. Esse crescimento decorre do aumento das rotas disponíveis para o tráfego de pacotes, o que, por sua vez, eleva a probabilidade de ocorrência de eventos de trânsito e de rearmazenamento.

Com base nesses dados, justifica-se a análise das curvas suavizadas de tempo de processamento do sistema de consultas em função do número de armazéns, conforme ilustrado na Figura 5. Nota-se que, com o aumento do número de armazéns envolvidos, o tempo de processamento também cresce. Por transitividade, infere-se que o incremento no número de eventos lidos no arquivo de entrada acarreta maior custo computacional para o processamento das consultas.

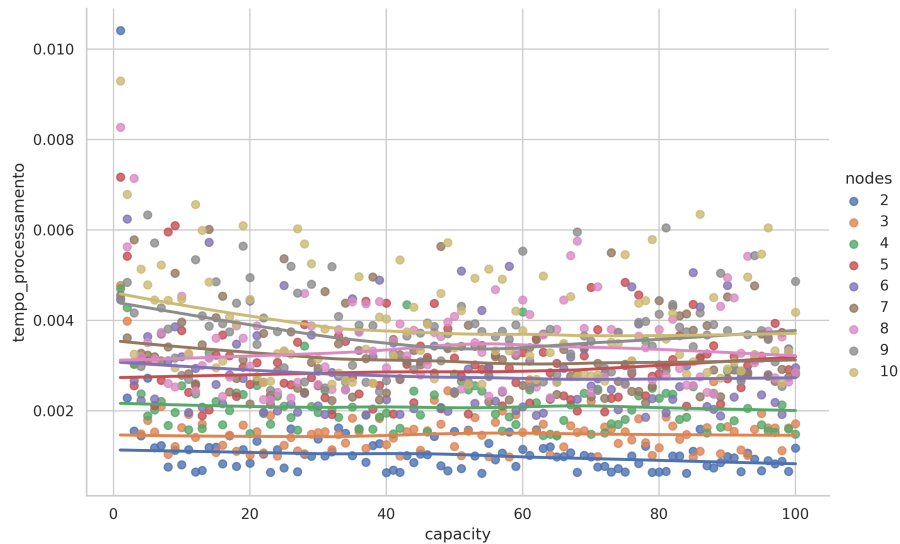


Figura 5: Tempo em função da capacidade para diferentes quantias de armazém.

6 Conclusão

Este trabalho lidou com o problema da recuperação eficiente de informações de eventos no sistema dos Armazéns Hanói, adotando como abordagem o uso de árvores AVL para estruturar os dados de pacotes e clientes, alvos de consulta. Desse modo, a proposta visou garantir rapidez nas buscas e inserções, mesmo sob grande volume de eventos.

A solução demonstrou ser eficiente tanto em termos de tempo quanto de memória, conforme evidenciado pela análise de complexidade e pelos experimentos realizados. A estrutura balanceada permitiu um desempenho estável e escalável para o sistema de consultas.

Ao longo do desenvolvimento, foi possível aplicar na prática conceitos como estruturas de dados dinâmicas, análise assintótica e modularização em C++. Entre os principais desafios, destacam-se o tratamento adequado dos eventos e a sincronização entre os arquivos de entrada gerados e os módulos de execução.

7 Bibliografia

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Algoritmos: Teoria e Prática*. 3ª Edição. Rio de Janeiro: Elsevier, 2012. (Tradução de: **Introduction to Algorithms**, MIT Press, 2009).
- Slides da disciplina DCC205/DCC221 - Estruturas de Dados, 2025/1.